

Final Project

David Koyrakh

Bellevue University

DSC-650: Big Data

Professor Nasheb Ismaily

November 19, 2025

Final Project

Introduction

For my final project, I customized and deployed a big data pipeline for moving an advertisement-performance dataset through NiFi, HDFS, Hive, Spark (via YARN), and HBase. The pipeline includes building and feeding the data through a Linear Regression model in order to understand the predictive power of certain advertising metrics against total revenue. This exercise demonstrates how the technologies work together in a realistic big data pipeline.

Dataset

I generated a synthetic dataset which contains metrics for 100 online advertising campaigns. It is hosted publicly in my GitHub repo here: https://github.com/dovkoy/big-data-650/blob/trunk/online_ads.csv

The dataset includes eight columns that measure different aspects of an advertisement's performance, such as ad spend, impressions, clicks, conversions, click-through rate (CTR), and revenue. I intentionally kept the dataset small (100 rows) to avoid overloading the VM's limited resources. The file required no complex cleaning, besides removing the header row that Hive initially ingested as a null record. Using this data allowed me to practice orchestrating the pipeline on a real-life business problem: how might online advertisement spending and performance metrics impact a company's total revenue?

Pipeline Overview

This pipeline begins with a NiFi flow for downloading the dataset from my public GitHub repo and writing it to HDFS. The data is then loaded into a Hive table for structured access. Next, Spark (running on YARN) reads the data from Hive, builds and evaluates a linear regression model on it, and outputs metrics such as RMSE and R^2 . These model results are then written to an HBase table using HappyBase. This completes the workflow of ingestion, storage, processing, and NoSQL persistence.

Issues Encountered

- My CSV file contained a header row and when initially loading the dataset, I did not include the parameter for skipping the header row. Thus, I had to manually remove the first “NULL” row that was created in the table.
- The master container did not have git or apt-get installed, so I had to clone the repo in the host VM and copy my Spark ML script into the container manually. The command I used to copy from the host to the Docker container was:
 - o `docker cp /home/dovid/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase/big-data-650/sparkml.py hadoop-hive-spark-hbase_master_1:/root/`

Screenshots

I started by loading my dataset into HDFS using the provided template. The template includes a processor for downloading a file from the public internet, passing it through a queue to another processor which uploads it with the correct filename and path, and then through another queue to the final processor, which writes the file to HDFS:

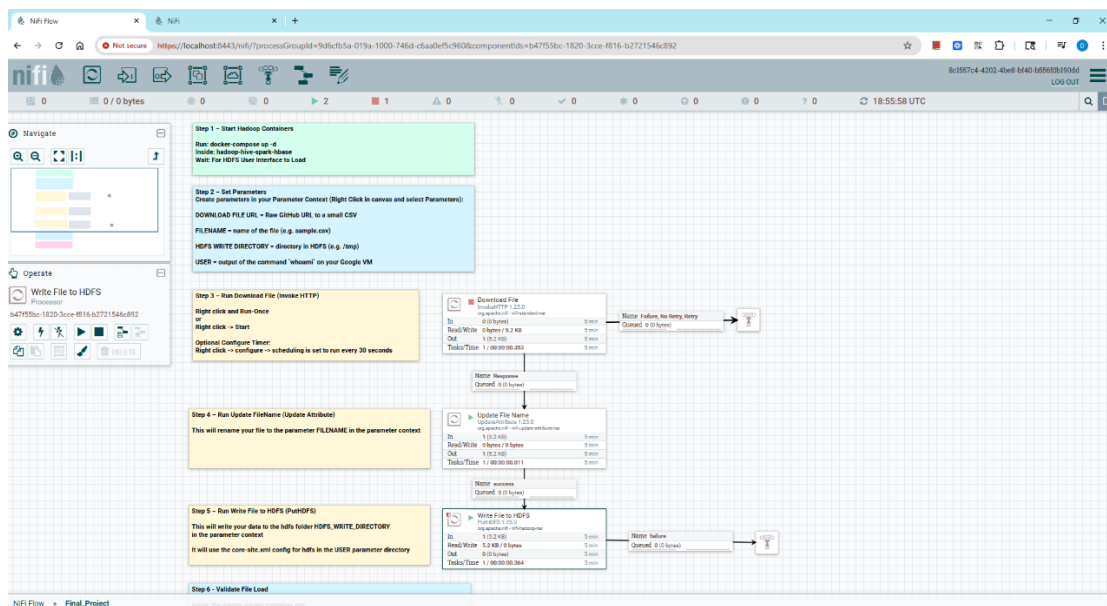


Figure 1: Provided NiFi template running to download my dataset and store it in HDFS.

I then opened the Docker container's master bash shell and verified the presence of my dataset in HDFS. The file correctly showed in the "tmp" directory of HDFS, and running "cat" on the file confirmed its contents are correct:

```
dovid@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
dovid@dsc-650-vm:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose exec master bash
bash-5.0# hdfs dfs -ls /tmp
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2025-11-19 18:54:15,726 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
drwx-wx-wx - root supergroup 0 2025-11-19 18:37 /tmp/hive
-rw-r--r-- 3 dovid supergroup 5324 2025-11-19 18:52 /tmp/online_ads.csv
bash-5.0# hdfs dfs -cat /tmp/online_ads.csv
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2025-11-19 18:54:44,480 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
campaign_id,ad_spend_usd,impressions,clicks,ctr,conversions,conversion_rate,revenue_usd
1,6574.55,282714,4912,0.01737,257,0.05232,25000
2,8975.71,452668,10857,0.02398,170,0.01566,14636.66
3,752.09,48028,1644,0.03423,186,0.11314,16623.51
4,8189.59,333970,13780,0.04126,1965,0.1426,25000
5,2608.37,226263,7157,0.03164,1008,0.14088,25000
6,2601.5,184044,9475,0.05149,1302,0.1374,25000
7,5587.32,441217,13097,0.02968,892,0.06809,25000
8,5948.09,323335,11230,0.03472,1514,0.13486,25000
9,1952.57,158923,6923,0.04355,1217,0.17579,25000
10,4573.39,285875,12579,0.04398,2155,0.1713,25000
11,8911.07,794160,29601,0.03727,5079,0.17153,25000
12,7341.82,593959,29108,0.04899,4179,0.14357,25000
13,8604.73,346330,11730,0.03387,2017,0.17201,25000
```

Figure 2: Terminal output confirming the presence and validity of my dataset as stored in HDFS.

In the next step, I defined a new table in Hive called “ads_raw” with corresponding columns that conform to the dataset’s types. I then loaded the CSV data into the table and selected all rows to preview and ensure that the data had been loaded correctly:

```
dovid@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
hive> CREATE TABLE ads_raw (
  >   campaign_id INT,
  >   ad_spend_usd DOUBLE,
  >   impressions INT,
  >   clicks INT,
  >   ctr DOUBLE,
  >   conversions INT,
  >   conversion_rate DOUBLE,
  >   revenue_usd DOUBLE
  > )
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ','
  > STORED AS TEXTFILE;
OK
Time taken: 1.127 seconds
hive> LOAD DATA INPATH '/tmp/online_ads.csv' INTO TABLE ads_raw;
Loading data to table default.ads_raw
OK
Time taken: 0.482 seconds
hive> select * from ads_raw;
OK
NULL      NULL      NULL      NULL      NULL      NULL      NULL      NULL
1         6574.55  282714   4912      0.01737  257      0.05232  25000.0
2         8975.71  452668   10857     0.02398  170      0.01566  14636.66
3         752.09   48028    1644      0.03423  186      0.11314  16623.51
4         8189.59  333970   13780     0.04126  1965     0.1426   25000.0
5         2608.37  226263   7157      0.03164  1008     0.14088  25000.0
6         2601.5   184044   9475      0.05149  1302     0.1374   25000.0
7         5587.32  441217   13097     0.02968  892      0.06809  25000.0
8         5948.09  323335   11230     0.03472  1514     0.13486  25000.0
9         1952.57  158923   6923      0.04355  1217     0.17579  25000.0
10        4573.39  285875   12579     0.04398  2155     0.1713   25000.0
11        8911.07  794160   29601     0.03727  5079     0.17153  25000.0
12        7341.82  593959   29108     0.04899  4179     0.14357  25000.0
13        8604.73  346330   11730     0.03387  2017     0.17201  25000.0
14        2013.77  100624   4707      0.04678  903      0.19176  25000.0
15        3338.43  271755   7903      0.02908  1243     0.15734  25000.0
16        1589.83  64439    3063      0.04754  592      0.19343  25000.0
```

Figure 3: Hive SQL commands showing table definition, data loading, and populated “ads_raw” table rows.

As shown in Figure 3, I accidentally created a NULL row, since I didn’t explicitly tell Hive to skip the CSV header row when loading the data. Thus, I decided to run an “INSERT OVERWRITE” statement to correct my error and ensure only valid data exists in the table:

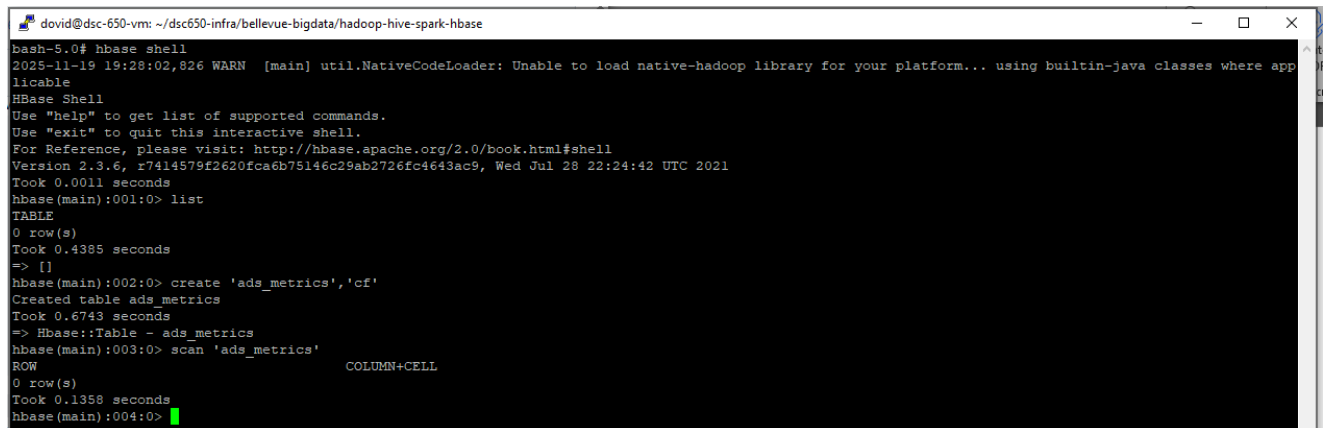
```
david@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
hive>
> INSERT OVERWRITE TABLE ads_raw
> SELECT * FROM ads_raw
> WHERE campaign_id IS NOT NULL;
Query ID = root_20251119191515_6714b50d-256e-4f2b-b597-63f38bd29980
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1763577353343_0007)

-----
VERTICES      MODE      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
-----
Map 1 ..... container  SUCCEEDED    1         1         0         0         0         0
Reducer 2 ..... container  SUCCEEDED    1         1         0         0         0         0
-----
VERTICES: 02/02  [=====>>>] 100%  ELAPSED TIME: 4.45 s
-----
Loading data to table default.ads_raw
OK
Time taken: 11.667 seconds
hive> select * from ads_raw
> ;
OK
1      6574.55 282714 4912    0.01737 257    0.05232 25000.0
2      8975.71 452668 10857   0.02398 170    0.01566 14636.66
3      752.09 48028 1644    0.03423 186    0.11314 16623.51
4      8189.59 333970 13780   0.04126 1965   0.1426 25000.0
5      2608.37 226263 7157    0.03164 1008   0.14088 25000.0
6      2601.5 184044 9475    0.05149 1302   0.1374 25000.0
7      5587.32 441217 13097   0.02968 892    0.06809 25000.0
8      5948.09 323335 11230   0.03472 1514   0.13486 25000.0
```

Figure 4: Deleting the accidental null row that was created from the CSV's header.

I chose INT for `campaign_id` because it is an incremental ID integer, DOUBLE for `ad_spend_usd` because it stores floating-point/decimal amounts in USD, INT for impressions because it stores whole-number counts of advertisement impressions, INT for clicks (same reason as impressions), DOUBLE for `ctr` (Click-Through Rate) because it is a rate expressed as a decimal, INT for conversions because it is a whole-number count of ad conversions, DOUBLE for `conversion_rate` since it is a rate expressed as a decimal, and DOUBLE for `revenue_usd` (same reason as `ad_spend_usd`).

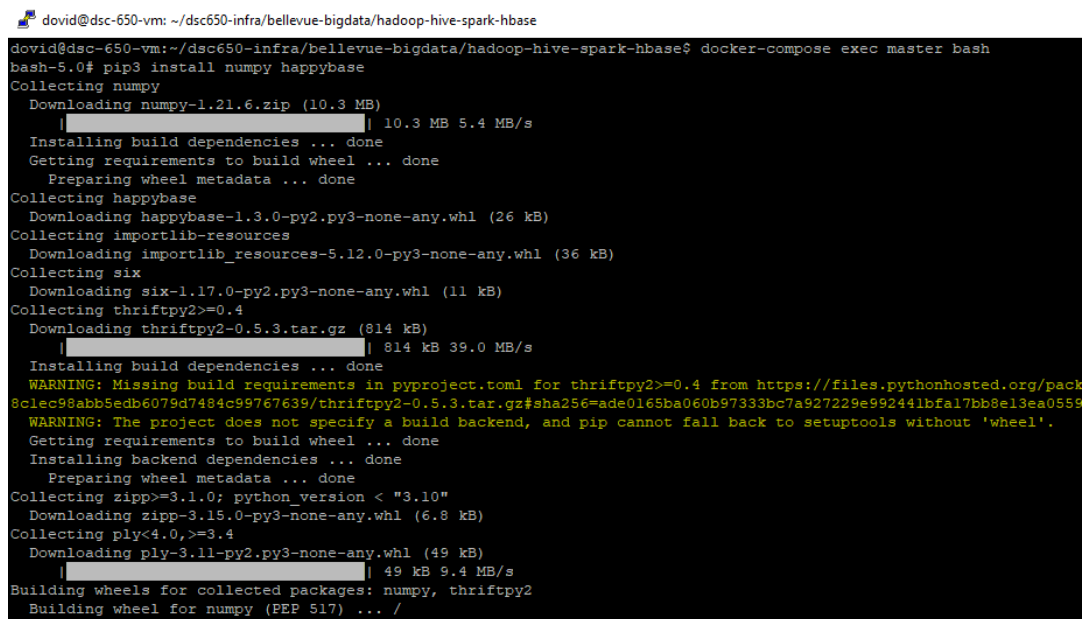
I then exited out of Hive and entered HBase shell. There, I create a table and column family for my result data called “ads_metrics” and “cf”, respectively:



```
david@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# hbase shell
2025-11-19 19:28:02,826 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.3.6, r7414579f2620fca6b75146c29ab2726fc4643ac9, Wed Jul 28 22:24:42 UTC 2021
Took 0.0011 seconds
hbase(main):001:0> list
TABLE
0 row(s)
Took 0.4385 seconds
=> []
hbase(main):002:0> create 'ads_metrics','cf'
Created table ads_metrics
Took 0.6743 seconds
=> Hbase::Table - ads_metrics
hbase(main):003:0> scan 'ads_metrics'
COLUMN+CELL
0 row(s)
Took 0.1358 seconds
hbase(main):004:0>
```

Figure 5: HBase commands to create my table and column family with confirmation.

Next, I went ahead and installed the required libraries for machine learning and data processing in Python. I installed these dependencies in both the master, worker1, and worker2 containers:



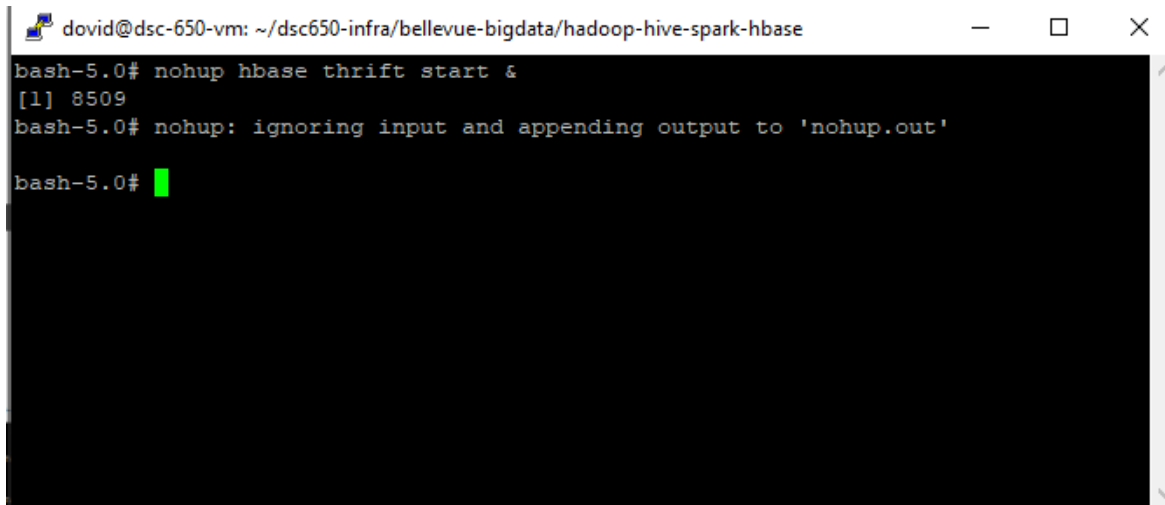
```
david@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
david@dsc-650-vm:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose exec master bash
bash-5.0# pip3 install numpy happybase
Collecting numpy
  Downloading numpy-1.21.6.zip (10.3 MB)
    |#####| 10.3 MB 5.4 MB/s
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing wheel metadata ... done
Collecting happybase
  Downloading happybase-1.3.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources
  Downloading importlib_resources-5.12.0-py3-none-any.whl (36 kB)
Collecting six
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting thriftpy2>=0.4
  Downloading thriftpy2-0.5.3.tar.gz (814 kB)
    |#####| 814 kB 39.0 MB/s
  Installing build dependencies ... done
  WARNING: Missing build requirements in pyproject.toml for thriftpy2>=0.4 from https://files.pythonhosted.org/packages/8c/lec98abb5eddb6079d7484c99767639/thriftpy2-0.5.3.tar.gz#sha256=ade0165ba060b97333bc7a927229e992441bf17bb8e13ea0559
  WARNING: The project does not specify a build backend, and pip cannot fall back to setuptools without 'wheel'.
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing wheel metadata ... done
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
    |#####| 49 kB 9.4 MB/s
Building wheels for collected packages: numpy, thriftpy2
  Building wheel for numpy (PEP 517) ... /
```

Figure 6: Installation of the required Python dependencies inside a Docker container.


```
david@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing wheel metadata ... done
Collecting happybase
  Downloading happybase-1.3.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources
  Downloading importlib_resources-5.12.0-py3-none-any.whl (36 kB)
Collecting six
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting thriftpy2>=0.4
  Downloading thriftpy2-0.5.3.tar.gz (814 kB)
  Installing build dependencies ... done
  WARNING: Missing build requirements in pyproject.toml for thriftpy2>=0.4 from https://files.pythonhosted.org/packages/64/d4/ef68e81e626dbce89d25fe728a538c1ec98abb5edb6079d7484c99767639/thriftpy2-0.5.3.tar.gz#sha256=ade0165ba060b97333bc7a927229e992441bf17bb8e13ea05590c2ec3551b17 (from happybase).
  WARNING: The project does not specify a build backend, and pip cannot fall back to setuptools without 'wheel'.
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing wheel metadata ... done
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
Building wheels for collected packages: numpy, thriftpy2
  Building wheel for numpy (PEP 517) ... done
  Created wheel for numpy: filename=numpy-1.21.6-cp37-cp37m-linux_x86_64.whl size=16644722 sha256=4cdd919455199198e2d3f8809770c451b47f663f0840f405ec8e5d3f7d58b1e0
  Stored in directory: /root/.cache/pip/wheels/4e/7e/9e/0fde042ccff2493994076dac9c3fbd78feb444c3bd94eb386a
  Building wheel for thriftpy2 (PEP 517) ... done
  Created wheel for thriftpy2: filename=thriftpy2-0.5.3-cp37-cp37m-linux_x86_64.whl size=1474144 sha256=210028ead1a5ffecbccd68c78eea011a5bb0b309860c147b9a8fbaf14508f1cd
  Stored in directory: /root/.cache/pip/wheels/c6/b0/28/a3eb930013c808961b4978078592e8d1d8b29ccce3149d8fe6
Successfully built numpy thriftpy2
Installing collected packages: numpy, zipp, importlib-resources, six, ply, thriftpy2, happybase
Successfully installed happybase-1.3.0 importlib-resources-5.12.0 numpy-1.21.6 ply-3.11 six-1.17.0 thriftpy2-0.5.3 zipp-3.15.0
bash-5.0#
```

Figure 7: Output confirming successful installation of the Python dependencies for Spark. This was repeated for the master, worker1, and worker2 containers.

Next, I went into the master container and started the Thrift server, using the “nohup ... &” syntax to ensure it continues running in the background, even after I exit the shell. This server creates a communication bridge between Spark and HBase. Specifically, the Thrift server listens for external events/requests and writes them to HBase.

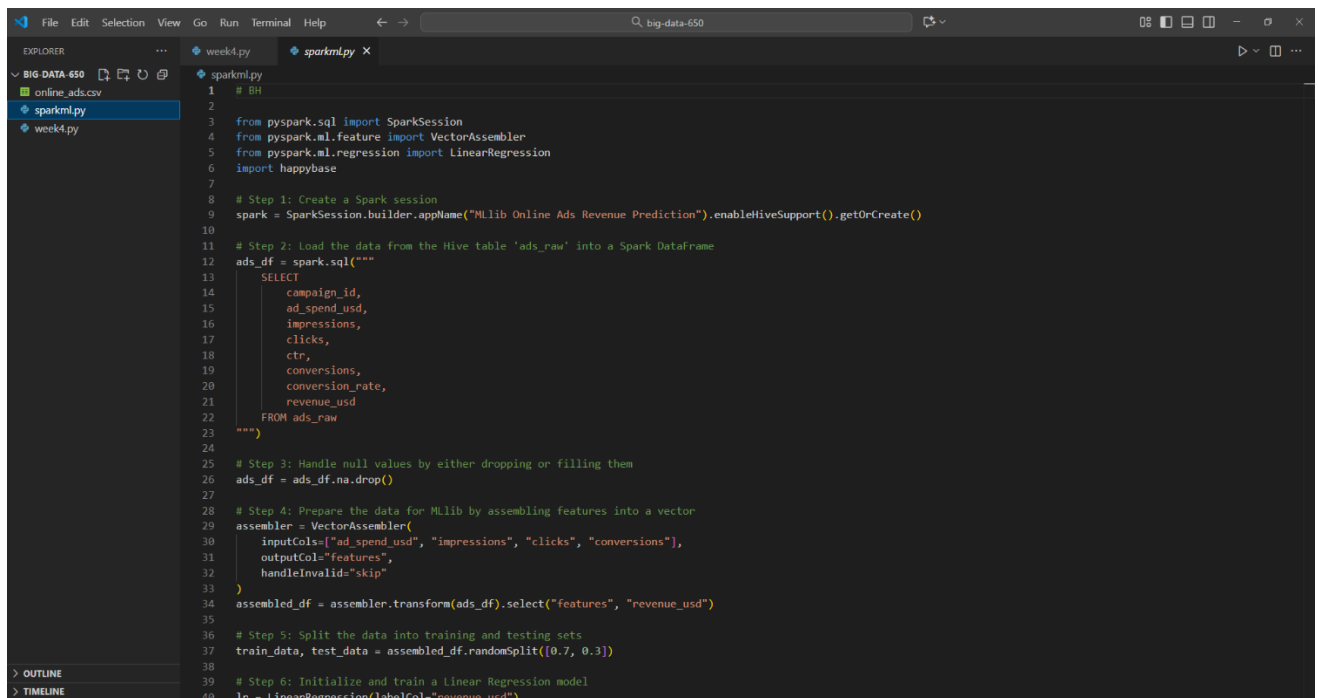
A terminal window titled 'dovid@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase'. The terminal shows the command 'bash-5.0# nohup hbase thrift start &' being entered. The output is '[1] 8509' followed by 'bash-5.0# nohup: ignoring input and appending output to 'nohup.out''. The prompt 'bash-5.0#' is shown again with a green cursor.

```
dovid@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# nohup hbase thrift start &
[1] 8509
bash-5.0# nohup: ignoring input and appending output to 'nohup.out'
bash-5.0#
```

Figure 8: Starting the Thrift server in the master container's background.

I then moved on to the Spark ML steps and edited the linear regression Spark code for my dataset. Then, I pushed the code to my public DSC-650 repo.

I chose linear regression because my dataset contains continuous advertising features, such as ad spend, impressions, and clicks. These variables directly influence my continuous target variable, revenue, which makes regression a strong modeling choice.



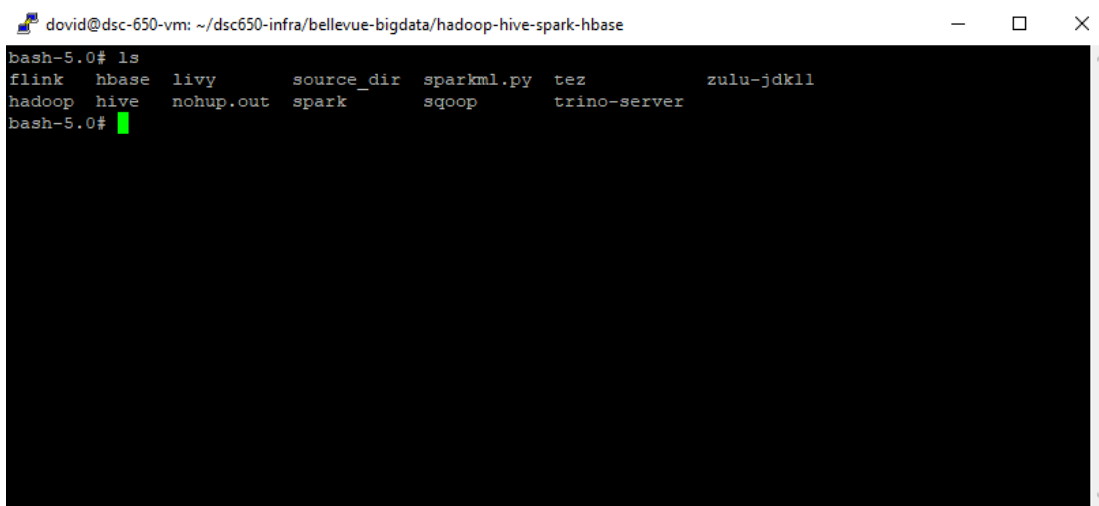
```

1  # BH
2
3  from pyspark.sql import SparkSession
4  from pyspark.ml.feature import VectorAssembler
5  from pyspark.ml.regression import LinearRegression
6  import happybase
7
8  # Step 1: Create a Spark session
9  spark = SparkSession.builder.appName("MLlib Online Ads Revenue Prediction").enableHiveSupport().getOrCreate()
10
11 # Step 2: Load the data from the Hive table 'ads_raw' into a Spark DataFrame
12 ads_df = spark.sql("""
13     SELECT
14         campaign_id,
15         ad_spend_usd,
16         impressions,
17         clicks,
18         ctr,
19         conversions,
20         conversion_rate,
21         revenue_usd
22     FROM ads_raw
23 """)
24
25 # Step 3: Handle null values by either dropping or filling them
26 ads_df = ads_df.na.drop()
27
28 # Step 4: Prepare the data for MLlib by assembling features into a vector
29 assembler = VectorAssembler(
30     inputCols=["ad_spend_usd", "impressions", "clicks", "conversions"],
31     outputCol="features",
32     handleInvalid="skip"
33 )
34 assembled_df = assembler.transform(ads_df).select("features", "revenue_usd")
35
36 # Step 5: Split the data into training and testing sets
37 train_data, test_data = assembled_df.randomSplit([0.7, 0.3])
38
39 # Step 6: Initialize and train a Linear Regression model
40 lr = LinearRegression(labelCol="revenue_usd")

```

Figure 9: Editing the “sparkml.py” file in VS Code to cater it for my dataset.

Back in my VM, I cloned the repo and copied the sparkml.py script into the master container’s default directory (/usr/program).



```

david@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# ls
flink  hbase  livy      source_dir  sparkml.py  tez      zulu-jdk11
hadoop  hive   nohup.out  spark       sqoop       trino-server
bash-5.0#

```

Figure 10: Confirmation that my Spark ML script was successfully copied into the VM master Docker container.

I ran the script using “spark-submit --master yarn --deploy-mode client sparkml.py”, waited for the job to finish, and opened the job’s tracking link:

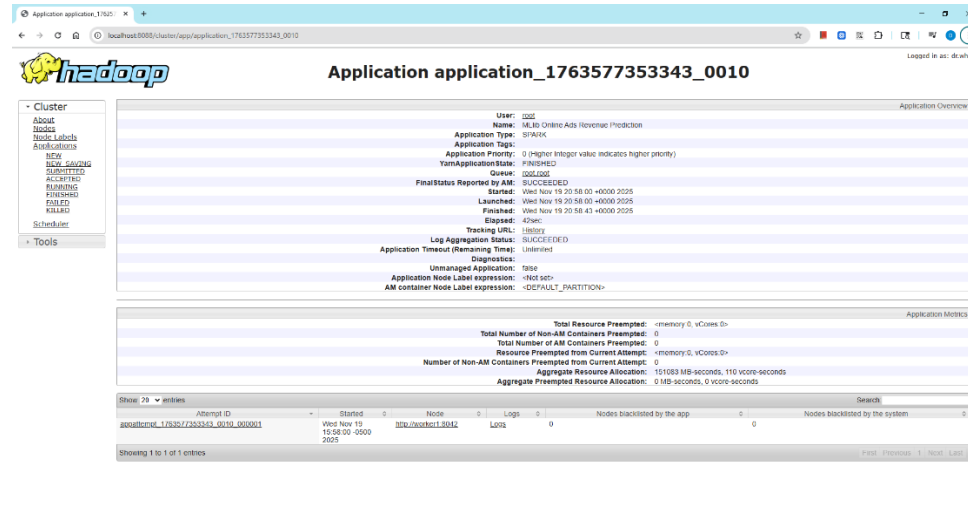
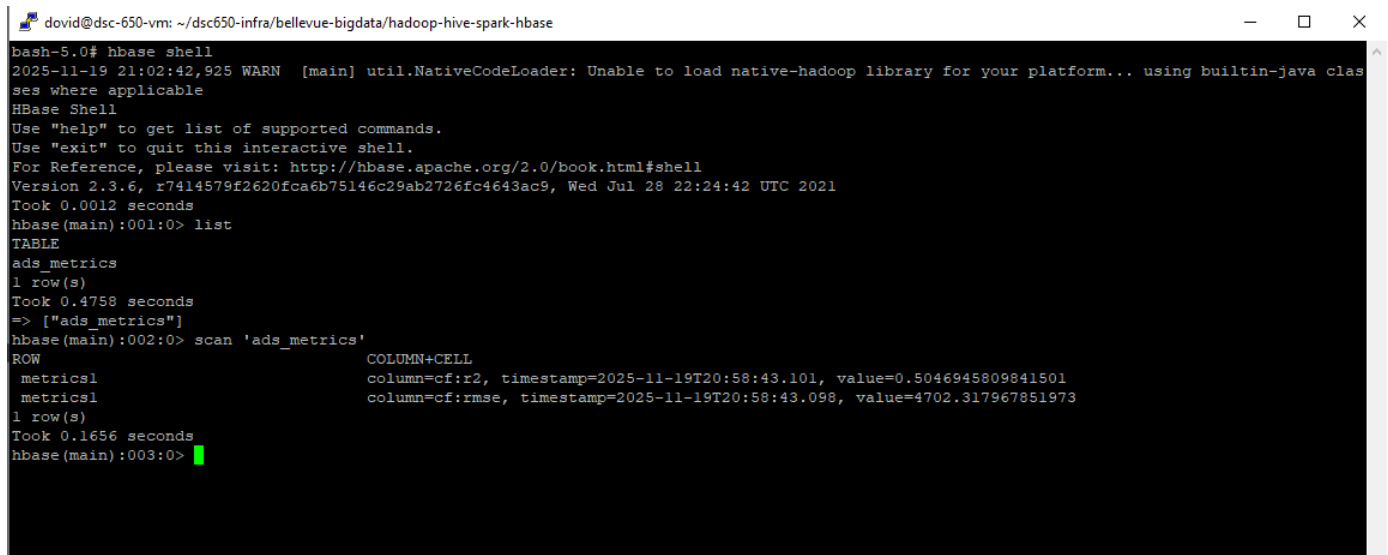


Figure 11: Hadoop UI open the job’s tracking link, showing it completed successfully.

```
david@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
35685 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.cluster.YarnScheduler - Adding task set 6.0 with 1 tasks
35687 [dispatcher-CoarseGrainedScheduler] INFO org.apache.spark.scheduler.TaskSetManager - Starting task 0.0 in stage 6.0 (TID 11, worker1,
executor 1, partition 0, NODE_LOCAL, 7336 bytes)
35690 [dispatcher-BlockManagerMaster] INFO org.apache.spark.storage.BlockManagerInfo - Added broadcast_12_piece0 in memory on worker1:7002
(size: 5.0 KiB, free: 346.1 MiB)
35703 [dispatcher-event-loop-6] INFO org.apache.spark.MapOutputTrackerMasterEndpoint - Asked to send map output locations for shuffle 1 to
172.28.1.2:42366
35711 [task-result-getter-3] INFO org.apache.spark.scheduler.TaskSetManager - Finished task 0.0 in stage 6.0 (TID 11) in 24 ms on worker1 (
executor 1) (1/1)
35711 [task-result-getter-3] INFO org.apache.spark.scheduler.cluster.YarnScheduler - Removed TaskSet 6.0, whose tasks have all completed, f
rom pool
35712 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - ResultStage 6 (count at LinearRegression.scala:933) finishe
d in 0.032 s
35713 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Job 4 is finished. Cancelling potential speculative or zomb
ie tasks for this job
35713 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.cluster.YarnScheduler - Killing all running tasks in stage 6: Stage finish
ed
35713 [Thread-5] INFO org.apache.spark.scheduler.DAGScheduler - Job 4 finished: count at LinearRegression.scala:933, took 0.217156 s
RMSE (Error): 6143.74063435407
R^2 (Fit Quality): 0.16337224186334853
Coefficients: [1.0254584815248018,-0.0023537122638005096,-0.0298848788767176,1.8681194908125363]
Intercept: 10537.872497800865
35806 [Thread-5] INFO org.apache.spark.SparkContext - Starting job: foreachPartition at /usr/program/sparkml.py:72
35808 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Got job 5 (foreachPartition at /usr/program/sparkml.py:72)
with 2 output partitions
35809 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Final stage: ResultStage 7 (foreachPartition at /usr/progra
m/sparkml.py:72)
35809 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Parents of final stage: List()
35809 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Missing parents: List()
35809 [dag-scheduler-event-loop] INFO org.apache.spark.scheduler.DAGScheduler - Submitting ResultStage 7 (PythonRDD[56] at foreachPartition
```

Figure 12: Printed output from job run with model evaluation metrics.

I went back into HBase and checked the job’s result (containing metrics about the Linear Regression model) by scanning the “ads_metrics” table.



```
david@dsc-650-vm: ~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase
bash-5.0# hbase shell
2025-11-19 21:02:42,925 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.3.6, r7414579f2620fca6b75146c29ab2726fc4643ac9, Wed Jul 28 22:24:42 UTC 2021
Took 0.0012 seconds
hbase(main):001:0> list
TABLE
ads_metrics
1 row(s)
Took 0.4758 seconds
=> ["ads_metrics"]
hbase(main):002:0> scan 'ads_metrics'
ROW
metrics1      column=cf:r2, timestamp=2025-11-19T20:58:43.101, value=0.5046945809841501
metrics1      column=cf:rmse, timestamp=2025-11-19T20:58:43.098, value=4702.317967851973
1 row(s)
Took 0.1656 seconds
hbase(main):003:0>
```

Figure 13: HBase scan output of the “ads_metrics” table, showing the stored job result from Spark ML.

Across two runs, the linear regression model produced RMSE values ranging from approximately \$4,700 to \$6,100 and R^2 values from 0.16 to 0.50. These figures indicate that the marketing metrics explain part of the variation in revenue, but not everything. The variance in RMSE and R^2 values across runs is most likely due to noise, different sets of data chosen randomly for the test/training splits, and the small sample size of 100 rows. My model performed moderately well all things considered, potentially reflecting realistic advertising behavior.

Code

- My Hive SQL code for defining and cleaning my table is shown above in Figures 3 and 4.
- My PySpark ML script is available here: <https://github.com/dovkoy/big-data-650/blob/trunk/sparkml.py>

Conclusion

In this final project, I deployed a big data pipeline to ingest online ad performance data and use it to build and evaluate a Machine Learning model. This was done by staging the dataset in Hive, training and evaluating a Linear Regression model in Spark MLlib, and recording its metrics in an HBase NoSQL table. This stack covers ingestion, storage, querying, processing, and persistence using real big data technologies.

With more time, I would experiment with another algorithm, such as Random Forest or a Deep Learning Neural Network. I would also explore automating dependency management to eliminate having to manually install the same packages in each node.

In conclusion, the final project gave me experience orchestrating multiple big data services in one pipeline. This reinforces what I learned throughout the course and program, especially when it comes to truly understanding and utilizing the power of real, big data technology stacks.